

Pertemuan 5

Kompleksitas

Algoritma

dan Big-O

Pemrograman untuk Ilmu Data
dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera
Semester Ganjil 2026/2027

Hari ini

- Mengubah kode menjadi hitungan langkah.
- Merumuskan $T(n)$, lalu mengambil orde dominan.
- Melihat contoh: penjumlahan, loop, pembagian dua, dan jumlah segitiga.
- Membandingkan beberapa algoritma anagram.
- Membaca hasil eksperimen Python dengan hati-hati.

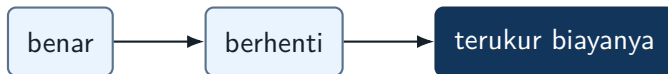
Dari M4 ke M5: program berhenti, lalu seberapa berat?

Pertanyaan dari M4

- Apakah program berhenti?
- Jika salah, bagian mana yang salah?
- Bagaimana menguji program dengan kasus batas?

Pertanyaan hari ini

- Jika dua program sama-sama benar, mana yang lebih hemat kerja?
- Ketika data membesar, bagian kode mana yang mulai mahal?
- Apa konsekuensinya untuk data analysis dan scientific computing?



Orde: dari ukuran biasa ke pertumbuhan fungsi

Dalam bahasa sehari-hari

- puluhan: kira-kira 10^1
- ratusan: kira-kira 10^2
- ribuan: kira-kira 10^3

Kita tidak sedang mencari angka persis, tetapi kelas besarnya.

Dalam algoritma

Misalnya ukuran input adalah n .

kerja program	orde
$7n + 3$	$O(n)$
$5n^2 + 27n + 1005$	$O(n^2)$
$2n + 26$	$O(n)$

Catatan matematika

Secara matematis, $f(n) = O(g(n))$ berarti setelah n cukup besar, $f(n)$ dapat dibatasi oleh konstanta kali $g(n)$.

Cara membaca banyak kerja dari sebuah program

Ukuran kerja program

Di sini $T(n)$ berarti fungsi jumlah kerja terhadap ukuran input n . Kata yang lebih tepat: **merumuskan** atau **menghitung** $T(n)$.

Langkah kerja

- 1 Tentukan ukuran input n .
- 2 Tentukan operasi yang dihitung.
- 3 Hitung berapa kali operasi itu dieksekusi.
- 4 Tulis $T(n)$ secara jelas.
- 5 Ambil suku yang dominan untuk Big-O.

Apa yang konstan?

- Assign awal seperti $s = 0$: konstan.
- `print` satu kali: konstan.
- Loop sebanyak n : bergantung n .
- Loop di dalam loop: biasanya bergantung hasil kali banyak iterasi.

Contoh 1: tujuan yang sama, cara kerja berbeda

Tujuan

Hitung

$$S(n) = 1 + 2 + \cdots + n.$$

Dua algoritma di bawah menghasilkan nilai yang sama.

```
def jumlah_loop(n):  
    s = 0  
    for i in range(1, n+1):  
        s = s + i  
    return s
```

Rumus tertutup

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

```
def jumlah_rumus(n):  
    return n*(n+1)/2
```

Contoh 1: tujuan yang sama, cara kerja berbeda

Tujuan

Hitung

$$S(n) = 1 + 2 + \dots + n.$$

Dua algoritma di bawah menghasilkan nilai yang sama.

```
def jumlah_loop(n):  
    s = 0  
    for i in range(1, n+1):  
        s = s + i  
    return s
```

Rumus tertutup

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

```
def jumlah_rumus(n):  
    return n*(n+1)/2
```

Pertanyaan kelas

Bagian mana yang diulang sebanyak n kali?

Contoh 1: menghitung $T(n)$ dari penjumlahan 1 sampai n

Versi loop

Hitung operasi utama: $s = s + i$.

$$\begin{aligned} T_{\text{loop}}(n) &= n \\ \Rightarrow T_{\text{loop}}(n) &= O(n). \end{aligned}$$

Jika ikut menghitung inisialisasi dan return, misalnya

$$T(n) = n + 2,$$

orde tetap $O(n)$.

Versi rumus

Jumlah operasi aritmatika tidak bertambah saat n membesar.

$$T_{\text{rumus}}(n) = c \Rightarrow O(1).$$

Contoh 1: menghitung $T(n)$ dari penjumlahan 1 sampai n

Versi loop

Hitung operasi utama: $s = s + i$.

$$\begin{aligned} T_{\text{loop}}(n) &= n \\ \Rightarrow T_{\text{loop}}(n) &= O(n). \end{aligned}$$

Jika ikut menghitung inisialisasi dan return, misalnya

$$T(n) = n + 2,$$

orde tetap $O(n)$.

Versi rumus

Jumlah operasi aritmatika tidak bertambah saat n membesar.

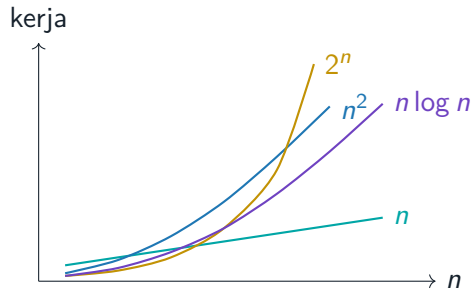
$$T_{\text{rumus}}(n) = c \Rightarrow O(1).$$

Pesan

Dua program dapat menghasilkan jawaban yang sama, tetapi biaya komputasinya berbeda.

"Keluarga" pertumbuhan yang sering muncul

Notasi	Gambaran kerja
$O(1)$	tetap
$O(\log n)$	dibagi dua berulang
$O(n)$	satu kali menyapu data
$O(n \log n)$	pecah-bagi lalu gabung
$O(n^2)$	pasangan elemen
$O(2^n)$	pilihan biner bertingkat
$O(n!)$	semua permutasi



Catatan

1. Untuk n kecil, beda kurva kadang belum terasa. Untuk n besar, orde mulai menentukan.
2. Kebergantungan algoritma terhadap n Biasanya sering disebut *time complexity*

Beberapa contoh asal orde

Pola komputasi	Contoh	Orde
Akses satu elemen	$x[10]$	$O(1)$
Satu penyapuan data	hitung total list	$O(n)$
Dua loop bersarang penuh	semua pasangan (i, j)	$O(n^2)$
Ukuran dibagi dua	binary search, $i = i//2$	$O(\log n)$
Sorting efisien	merge sort, TimSort rata-rata	$O(n \log n)$
Semua subset	pilih/tidak pilih tiap elemen	$O(2^n)$
Semua susunan	semua permutasi karakter	$O(n!)$

Contoh 2: Algoritma membagi dua hingga habis

```
def bagi_dua(n):  
    i = n  
    while i > 0:  
        i = i // 2
```

Telusuri untuk $n = 40$

$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0.$

Contoh 2: Algoritma membagi dua hingga habis

```
def bagi_dua(n):  
    i = n  
    while i > 0:  
        i = i // 2
```

Telusuri untuk $n = 40$

$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0.$

Setelah k kali:

$$\frac{n}{2^k} < 1.$$

Maka

$$2^k > n \Rightarrow k > \log_2 n.$$

Contoh 2: Algoritma membagi dua hingga habis

```
def bagi_dua(n):  
    i = n  
    while i > 0:  
        i = i // 2
```

Telusuri untuk $n = 40$

$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0.$

Setelah k kali:

$$\frac{n}{2^k} < 1.$$

Maka

$$2^k > n \Rightarrow k > \log_2 n.$$

Kesimpulan

Jumlah iterasi bertambah lambat:
 $O(\log n).$

Latihan kelas: tiga potongan kode

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

A

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

B

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

C

Latihan kelas: tiga potongan kode

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

A

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

B

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

C

Petunjuk

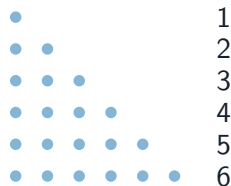
A: pasangan (i, j) . B: dua sapuan berurutan. C: ukuran dibagi dua setiap iterasi.

Contoh 3: jumlah segitiga

```
s = 0
for i in range(n):
    for j in range(i + 1):
        s += 1
```

Tujuan

Berapa kali `s += 1` dieksekusi?

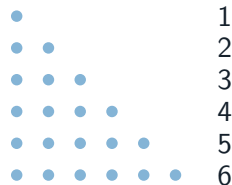


Contoh 3: jumlah segitiga

```
s = 0
for i in range(n):
    for j in range(i + 1):
        s += 1
```

Tujuan

Berapa kali `s += 1` dieksekusi?



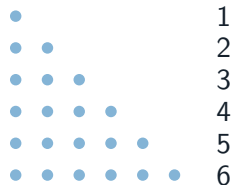
$$T(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Contoh 3: jumlah segitiga

```
s = 0
for i in range(n):
    for j in range(i + 1):
        s += 1
```

Tujuan

Berapa kali `s += 1` dieksekusi?



$$T(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

$$T(n) = \frac{1}{2}n^2 + \frac{1}{2}n \Rightarrow O(n^2).$$

Contoh 4: menghitung bagian yang bergantung pada n

```
a = 5
b = 6
c = 10
for i in range(n):
    for j in range(n):
        x = i*i
        y = j*j
        z = i*j
for k in range(n):
    w = a*k + 45
    v = b*b
d = 33
```

Contoh 4: menghitung bagian yang bergantung pada n

```
a = 5
b = 6
c = 10
for i in range(n):
    for j in range(n):
        x = i*i
        y = j*j
        z = i*j
for k in range(n):
    w = a*k + 45
    v = b*b
d = 33
```

$$T(n) = 3 + 3n^2 + 2n + 1 = 3n^2 + 2n + 4.$$

Contoh 4: menghitung bagian yang bergantung pada n

```
a = 5
b = 6
c = 10
for i in range(n):
    for j in range(n):
        x = i*i
        y = j*j
        z = i*j
for k in range(n):
    w = a*k + 45
    v = b*b
d = 33
```

$$T(n) = 3 + 3n^2 + 2n + 1 = 3n^2 + 2n + 4.$$

Dominan: $3n^2$, koefisien diabaikan untuk orde, sehingga $O(n^2)$.

Benchmark waktu: berguna, tetapi bukan definisi Big-O

Benchmark menjawab

- Program ini berjalan berapa detik di komputer ini?
- Apakah implementasi A lebih cepat daripada B pada ukuran data tertentu?
- Apakah dugaan orde tampak pada data eksperimen?

Yang memengaruhi waktu

- mesin: processor, memori, sistem operasi;
- bahasa dan interpreter/compiler;
- library yang digunakan;
- proses lain di komputer;
- ukuran dan bentuk data.

Prinsip

Big-O berasal dari hitungan langkah terhadap n . Benchmark dipakai untuk memeriksa dan membandingkan implementasi.

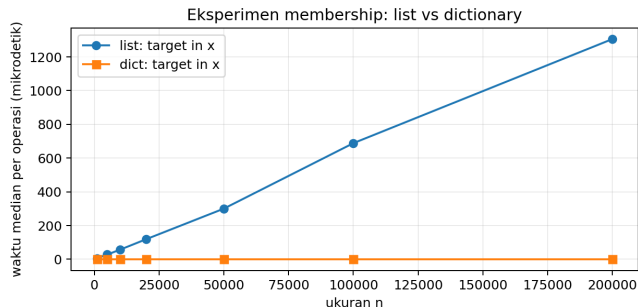
Eksperimen Python: membership pada list dan dictionary

```
# ide eksperimen
x = list(range(n))
target = n-1
target in x

x = {i: None for i in range(n)}
target in x
```

Dugaan sebelum menjalankan

- Pada list, pencarian dapat menyapu banyak elemen.
- Pada dictionary, pencarian key rata-rata hampir tetap.



Operasi list yang sering manipu

Operasi	Orde	Mengapa penting
<code>x[i]</code>	$O(1)$	akses langsung ke posisi
<code>x.append(v)</code>	$O(1)$	menambah di belakang biasanya murah
<code>x.pop()</code>	$O(1)$	ambil elemen terakhir
<code>x.pop(0)</code>	$O(n)$	elemen lain bergeser
<code>v in x</code>	$O(n)$	bisa perlu menyapu list
<code>x.sort()</code>	$O(n \log n)$	sorting efisien tetapi tidak gratis

Operasi list yang sering manipu

Operasi	Orde	Mengapa penting
<code>x[i]</code>	$O(1)$	akses langsung ke posisi
<code>x.append(v)</code>	$O(1)$	menambah di belakang biasanya murah
<code>x.pop()</code>	$O(1)$	ambil elemen terakhir
<code>x.pop(0)</code>	$O(n)$	elemen lain bergeser
<code>v in x</code>	$O(n)$	bisa perlu menyapu list
<code>x.sort()</code>	$O(n \log n)$	sorting efisien tetapi tidak gratis

Self check

Mana yang tidak $O(1)$: `pop(0)`, `pop()`, `append()`, atau `x[10]`?

Operasi list yang sering manipu

Operasi	Orde	Mengapa penting
<code>x[i]</code>	$O(1)$	akses langsung ke posisi
<code>x.append(v)</code>	$O(1)$	menambah di belakang biasanya murah
<code>x.pop()</code>	$O(1)$	ambil elemen terakhir
<code>x.pop(0)</code>	$O(n)$	elemen lain bergeser
<code>v in x</code>	$O(n)$	bisa perlu menyapu list
<code>x.sort()</code>	$O(n \log n)$	sorting efisien tetapi tidak gratis

Self check

Mana yang tidak $O(1)$: `pop(0)`, `pop()`, `append()`, atau `x[10]`?

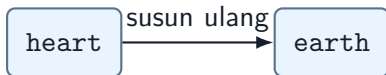
Petunjuk

Cari operasi yang membuat banyak elemen harus berpindah atau diperiksa.

Kasus utama: apakah dua kata anagram?

Definisi kerja

Dua string disebut anagram jika keduanya memakai karakter yang sama dengan banyak kemunculan yang sama, hanya urutannya berbeda.



Tiga strategi yang akan dibandingkan

- 1 Cek satu per satu dan tandai yang sudah dipakai.
- 2 Urutkan dua string, lalu bandingkan.
- 3 Hitung frekuensi setiap huruf, lalu bandingkan penghitungnya.

Anagram A: cek dan tandai

```
def anagram_A(s1, s2):  
    data = list(s2)  
    for ch in s1:  
        ketemu = False  
        for j in range(len(data)):  
            if ch == data[j]:  
                data[j] = None  
                ketemu = True  
                break  
        if not ketemu:  
            return False  
    return True
```

heart: cari h



Anagram A: cek dan tandai

```
def anagram_A(s1, s2):  
    data = list(s2)  
    for ch in s1:  
        ketemu = False  
        for j in range(len(data)):  
            if ch == data[j]:  
                data[j] = None  
                ketemu = True  
                break  
        if not ketemu:  
            return False  
    return True
```

heart: cari h



tandai

Pertanyaan

Jika huruf yang dicari baru ketemu di akhir list, berapa elemen yang diperiksa?

Anagram A: dari hitungan ke $O(n^2)$

Ukuran input

Misalkan panjang string adalah n .

Pada kasus yang berat

- Huruf pertama bisa memeriksa sampai n posisi.
- Huruf kedua bisa memeriksa sampai $n - 1$ posisi yang tersisa.
- Dan seterusnya.

$$T(n) = n + (n - 1) + \dots + 1.$$

Anagram A: dari hitungan ke $O(n^2)$

Ukuran input

Misalkan panjang string adalah n .

Pada kasus yang berat

- Huruf pertama bisa memeriksa sampai n posisi.
- Huruf kedua bisa memeriksa sampai $n - 1$ posisi yang tersisa.
- Dan seterusnya.

$$T(n) = n + (n - 1) + \dots + 1.$$

$$T(n) = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n.$$

Anagram A: dari hitungan ke $O(n^2)$

Ukuran input

Misalkan panjang string adalah n .

Pada kasus yang berat

- Huruf pertama bisa memeriksa sampai n posisi.
- Huruf kedua bisa memeriksa sampai $n - 1$ posisi yang tersisa.
- Dan seterusnya.

$$T(n) = n + (n - 1) + \dots + 1.$$

$$T(n) = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n.$$

$$T(n) = O(n^2).$$

Anagram B: urutkan, lalu bandingkan

```
def anagram_B(s1, s2):  
    a = sorted(s1)  
    b = sorted(s2)  
    return a == b
```

heart → aehrt earth → aehrt

Hitung bagian kerjanya

- Mengurutkan string pertama.
- Mengurutkan string kedua.
- Membandingkan hasil urutan.

Anagram B: urutkan, lalu bandingkan

```
def anagram_B(s1, s2):  
    a = sorted(s1)  
    b = sorted(s2)  
    return a == b
```

heart → aehrt earth → aehrt

Hitung bagian kerjanya

- Mengurutkan string pertama.
- Mengurutkan string kedua.
- Membandingkan hasil urutan.

Jika sorting yang dipakai $O(n \log n)$, maka

$$T(n) = 2 O(n \log n) + O(n).$$

Anagram B: urutkan, lalu bandingkan

```
def anagram_B(s1, s2):  
    a = sorted(s1)  
    b = sorted(s2)  
    return a == b
```

heart $\rightarrow$ aehrt earth $\rightarrow$ aehrt

Hitung bagian kerjanya

- Mengurutkan string pertama.
- Mengurutkan string kedua.
- Membandingkan hasil urutan.

Jika sorting yang dipakai $O(n \log n)$, maka

$$T(n) = 2 O(n \log n) + O(n).$$

Suku dominan: $O(n \log n)$.

Anagram C: hitung frekuensi

```
def anagram_C(s1, s2):  
    c1 = [0]*26  
    c2 = [0]*26  
    for ch in s1:  
        c1[ord(ch)-ord('a')] += 1  
    for ch in s2:  
        c2[ord(ch)-ord('a')] += 1  
    return c1 == c2
```

a	b	c	d	e	f	g	h	i	r	t
1	0	0	0	1	0	0	1	0	1	1

Ide

Karakter tidak dicari satu per satu. Setiap karakter langsung menambah penghitungnya.

Anagram C: mengapa $O(n)$?

Bagian yang dihitung

- Menyapu s1: n langkah.
- Menyapu s2: n langkah.
- Membandingkan 26 penghitung: 26 langkah.

$$T(n) = n + n + 26 = 2n + 26.$$

Anagram C: mengapa $O(n)$?

Bagian yang dihitung

- Menyapu s1: n langkah.
- Menyapu s2: n langkah.
- Membandingkan 26 penghitung: 26 langkah.

$$T(n) = n + n + 26 = 2n + 26.$$

$$T(n) = O(n).$$

Anagram C: mengapa $O(n)$?

Bagian yang dihitung

- Menyapu s1: n langkah.
- Menyapu s2: n langkah.
- Membandingkan 26 penghitung: 26 langkah.

$$T(n) = n + n + 26 = 2n + 26.$$

$$T(n) = O(n).$$

Trade-off

Lebih cepat, tetapi memakai memori tambahan untuk penghitung.

Tiga algoritma anagram: ringkas

Strategi	Kerja utama	$T(n)$ kasar	Orde
Cek dan tandai	pencarian bersarang	$\frac{1}{2}n^2 + \frac{1}{2}n$	$O(n^2)$
Sort dan bandingkan	sorting dua kali + bandingkan	$2n \log n + n$	$O(n \log n)$
Hitung frekuensi	dua sapuan + 26 pembandingan	$2n + 26$	$O(n)$

Tiga algoritma anagram: ringkas

Strategi	Kerja utama	$T(n)$ kasar	Orde
Cek dan tandai	pencarian bersarang	$\frac{1}{2}n^2 + \frac{1}{2}n$	$O(n^2)$
Sort dan bandingkan	sorting dua kali + bandingkan	$2n \log n + n$	$O(n \log n)$
Hitung frekuensi	dua sapuan + 26 pembandingan	$2n + 26$	$O(n)$

Diskusi kelas

Mana yang paling mudah ditulis? Mana yang paling hemat waktu? Mana yang memakai ruang tambahan?

Self check: baca kode, bukan tebak notasi

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Self check: baca kode, bukan tebak notasi

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Petunjuk

Untuk setiap nilai i , loop j berjalan n kali. Ada n pilihan i .

Self check: baca kode, bukan tebak notasi

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Petunjuk

Untuk setiap nilai i , loop j berjalan n kali. Ada n pilihan i .

$$T(n) = n \cdot n = n^2 \quad \Rightarrow \quad O(n^2).$$

Self check: dua loop berurutan

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

Self check: dua loop berurutan

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

Petunjuk

Loop kedua tidak berada di dalam loop pertama. Kerjanya dijumlahkan, bukan dikalikan.

Self check: dua loop berurutan

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

Petunjuk

Loop kedua tidak berada di dalam loop pertama. Kerjanya dijumlahkan, bukan dikalikan.

$$T(n) = n + n = 2n \Rightarrow O(n).$$

Self check: dibagi dua

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

Self check: dibagi dua

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

Petunjuk

Tulis nilai i setelah setiap iterasi: $n, n/2, n/4, n/8, \dots$

Self check: dibagi dua

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

Petunjuk

Tulis nilai i setelah setiap iterasi: $n, n/2, n/4, n/8, \dots$

Berhenti ketika $n/2^k < 1$, sehingga $k > \log_2 n$.

$$T(n) = O(\log n).$$

Yang harus bisa dilakukan

- 1 Mengurutkan orde pertumbuhan: 1 , $\log n$, n , $n \log n$, n^2 , n^3 , 2^n , $n!$.
- 2 Menghitung $T(n)$ dari potongan kode.
- 3 Membedakan loop bersarang dan loop berurutan.
- 4 Menjelaskan mengapa pembagian dua menghasilkan $\log n$.
- 5 Membandingkan algoritma yang sama-sama benar, termasuk kasus anagram.
- 6 Membaca benchmark sebagai bukti empiris, bukan definisi matematis.

- 1 Hitung $T(n)$ dan Big-O untuk kode jumlah segitiga.
- 2 Jelaskan dengan kalimat sendiri mengapa `pop(0)` lebih mahal daripada `pop()`.
- 3 Buat contoh dua algoritma yang menghasilkan output sama, tetapi satu $O(n)$ dan satu $O(1)$.
- 4 Untuk anagram, tuliskan satu alasan memilih strategi hitung frekuensi dan satu konsekuensinya.

Cara menjawab

Tulis proses hitungnya. Jangan hanya menulis notasi Big-O.

- Bradley N. Miller dan David L. Ranum. *Problem Solving with Algorithms and Data Structures Using Python*. Release 3.0. Bab 2: Algorithm Analysis.
- Lembar Kerja Mandiri M5: Kompleksitas Algoritma Big-O.